



Sistemas Distribuidos I (75.74)

Multitasking y Comunicaciones

Conceptos. Mecanismos de sincronización. IPCs. Problemas clásicos. Comunicaciones.

Docentes

- | | | |
|---------------------|--------------------|-------------------|
| ■ Pablo D. Roca | ■ Nicolás Zulaica | ■ Máximo Gismondi |
| ■ Gabriel Robles | ■ Franco Papa | ■ Camila Sebellín |
| ■ Franco Barreneche | ■ Manuel Reberendo | |

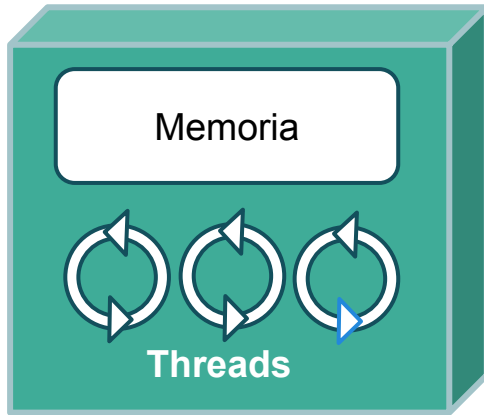


- **Conceptos**
- Mecanismos de sincronización
- IPCs
- Problemas clásicos
- Comunicaciones

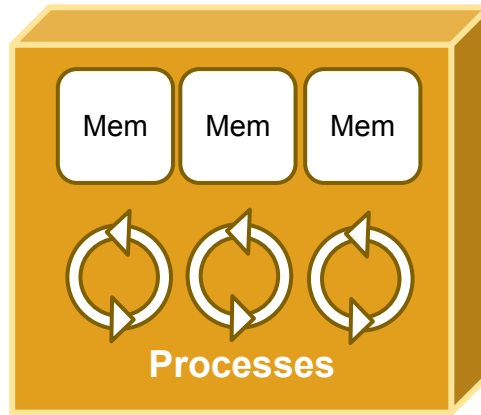


Introducción | Modelos de Multiprogramación

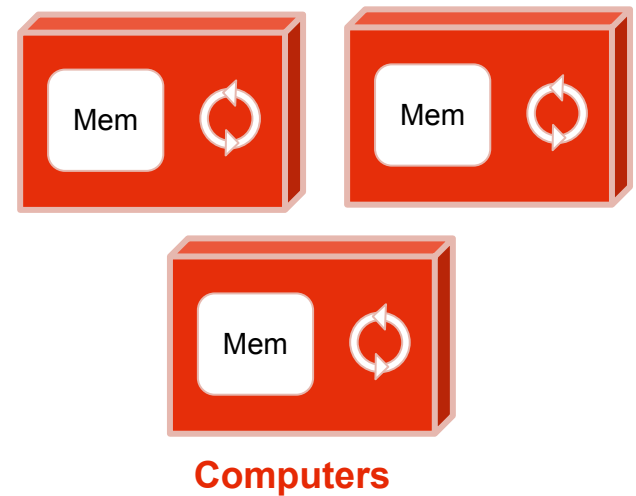
Multi-threading



Multi-processing



Multi-computing





Recursos Compartidos

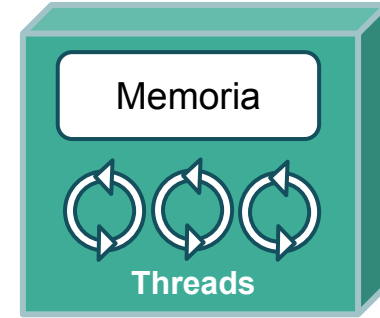
- Heap
- Data Segment
- File Descriptors
- Code Segment (read-only)

Sincronización

- Soporte threading del SO (pthread-mutex, etc)
- Soporte threading del *runtime* (threads Java, .Net, etc)
- Inter Process Communication (IPC)

Características clave

- Sencillo compartir información entre threads.
- Alto acoplamiento entre componentes del sistema.
- Escasa estabilidad => 1 thread defectuoso afecta todo el sistema.
- Escalabilidad muy limitada.





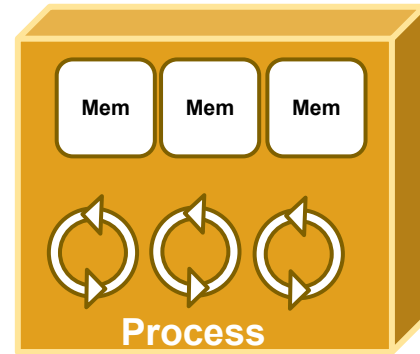
Conceptos | Multiprocessing

Recursos Compartidos

- Code Segment (read-only)
- File Descriptors (abiertos pre-fork)

Sincronización

- InterProcess Communication (IPCs):
 - Signals
 - Pipes / Fifos
 - Queues
 - Shared Memory
 - Semáforos
 - Locks
 - Sockets



Características clave

- No es trivial compartir información entre procesos.
- Componentes separados, en general simples.
- Más escalable y más estable que multi-threading.
- Sin tolerancia a fallos de hardware, sistema operativo, etc.



Recursos Compartidos

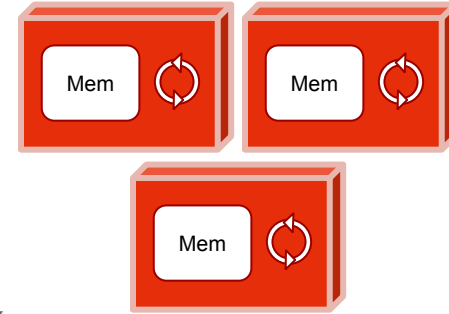
- Ninguno

Sincronización

- Mensajes Ad-Hoc entre Computadoras
=> necesidad de implementar mecanismos de sincronización.

Características clave

- Comunicación de red => problemas por limitaciones de ancho de banda, latencia y pérdida de mensajes.
- Comunicación entre procesos: compleja y central al diseño del sistema.
- Alta escalabilidad y tolerantes a fallos.





Safety properties (siempre verdadera - invariante)

- Exclusión mutua
- Ausencia de deadlocks

Liveness properties (eventualmente verdadera)

- Ausencia de starvation
- Fairness





Asegurar estado *safety* de propiedades de un sistema se transforma en un pilar de la teoría de concurrencia.

Basada en Algoritmos

- Sin existencia de abstracciones especiales.
- Condiciones lógicas simples para asegurar el cumplimiento de cierta *Critical Section*.

Basada en Abstracciones

- Basada en abstracciones provistas por el SO.
- Permite construir mecanismos compuestos por combinaciones de las mismas.



Conceptos | Basados en Algoritmos

- **Busy-Waiting:** responsable de la mayoría de los problemas de performance en sistemas concurrentes.
- **Spin-lock:** caso más simple de Busy-Wait (`while (flag);`)
- **Algoritmos de espera:** Dekker, Lamport (del panadero), Peterson, etc.

Ej. Algoritmo de Peterson p/dos procesos:

```
bool flag[2] = {false, false};
int turn;
P0:    flag[0] = true;
        turn = 1;
        while (flag[1] && turn == 1);
        /* critical section code */
        flag[0] = false;
```

```
P1:    flag[1] = true;
        turn = 0;
        while (flag[0] && turn == 0) ;
        /* critical section code */
        flag[1] = false;
```



Conceptos | Basados en Abstracciones

- **Operaciones atómicas:** Mecanismos provistos por un lenguaje para actualizar variables/objetos sin utilizar mecanismos de sincronización
 - **Contadores atómicos de tipos POD (int, char, double, etc.)**
 - **CAS (Compare and Swap):** Operación por excelencia para actualizar contenedores de forma segura en ambientes multithreading.

```
CAS (Pseudocódigo):  
function cas(p : *int, old : int, new  
: int) returns bool {  
    if *p ≠ old {  
        return false  
    }  
    *p ← new  
    return true  
}
```

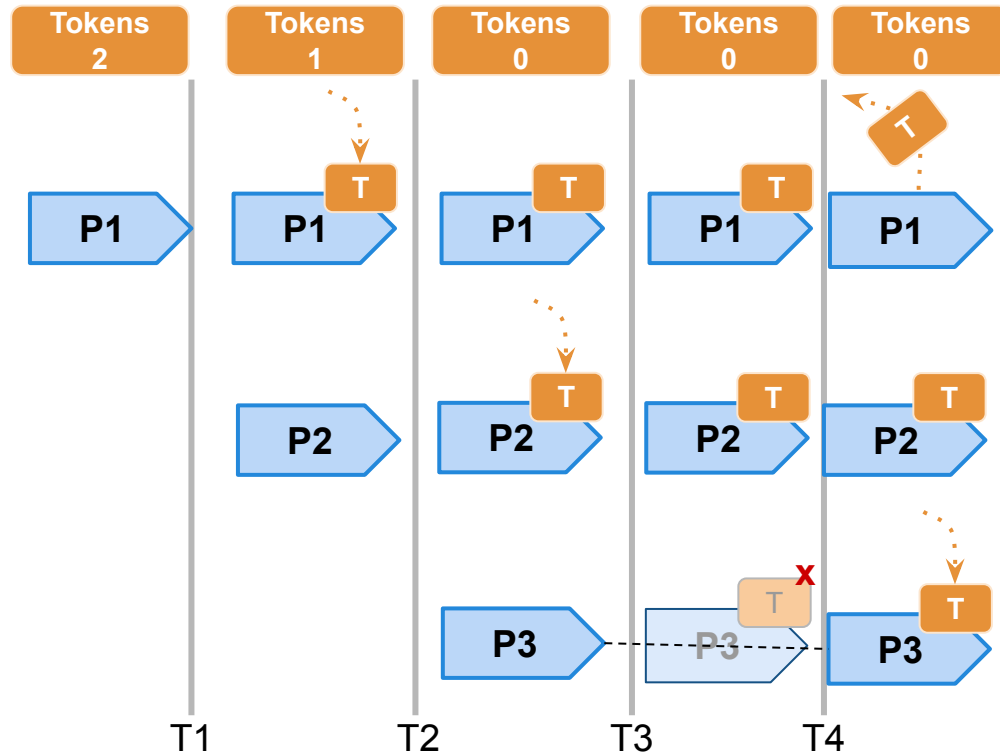
```
Ejemplo: CAS de objetos en Golang  
  
package atomic  
  
func CompareAndSwapUint32(  
    addr *uint32,  
    old uint32,  
    new uint32,  
    ) (swapped bool)
```



- Conceptos
- **Mecanismos de sincronización**
- IPCs
- Problemas clásicos
- Comunicaciones



Mecanismos de Sincronización | Semáforos





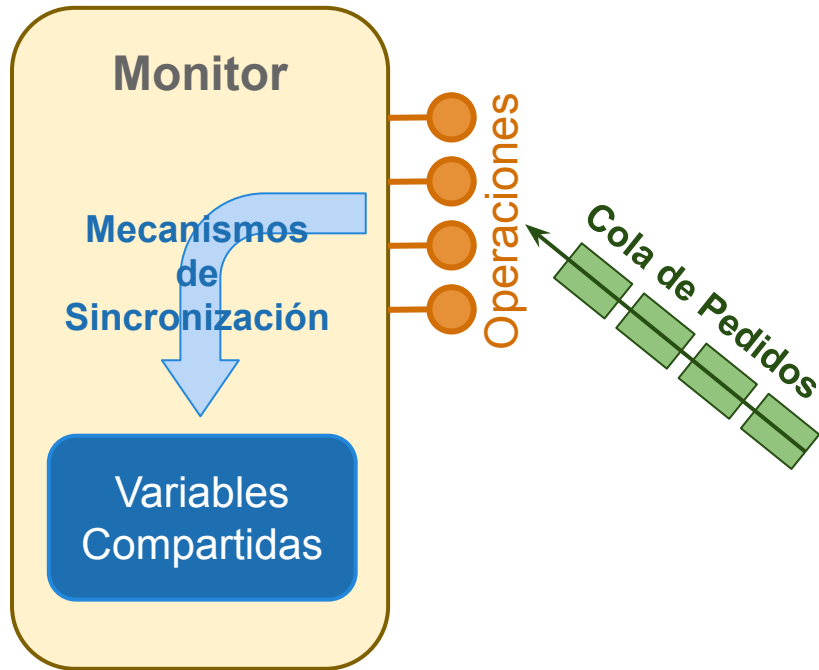
Mecanismos de Sincronización | Semáforos

- Variable entera utilizada para acceder a *recursos compartidos* (e.g. *Shared Mem*)
- El mismo queda definido por los valores que puede adoptar (e.g. $S = \{0,1,2\}$)
- Operaciones válidas:
 - signal (P): Incrementa el valor de S
 - wait (V): Decrementa el valor de S
- Mutex ($S = \{0,1\}$)
 - Utilizado para acceder a *secciones críticas*



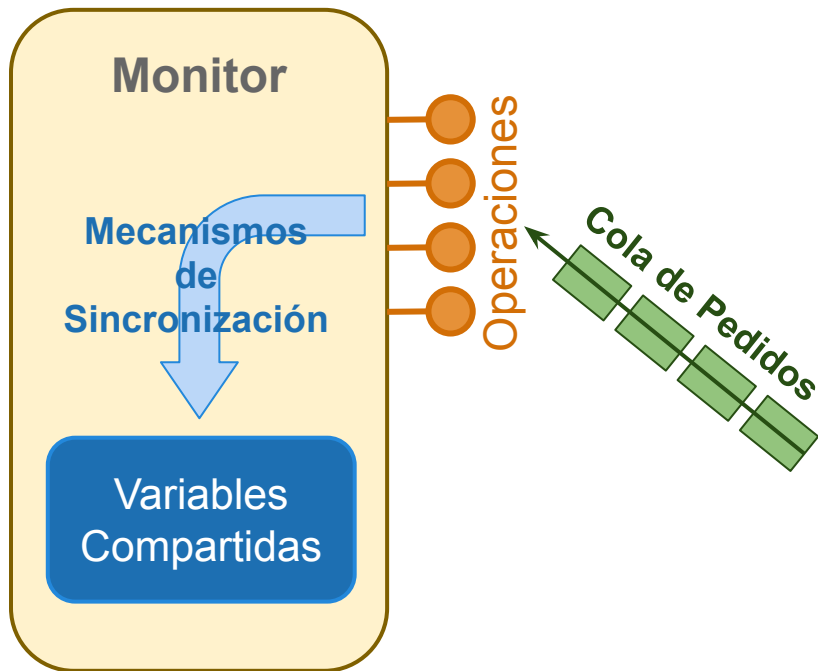


Mecanismos de Sincronización | Monitores





Mecanismos de Sincronización | Monitores



```
monitor class Account:
    private int balance = 0
    public method bool withdraw(int
amount):
    if balance < amount:
        return false
    else:
        balance = balance - amount
        return true

    public method deposit(int amount):
        balance := balance + amount
```



Mecanismos de Sincronización | Cond. Variables

- Ejemplo práctico de un monitor
- Mutex debe ser adquirido antes de realizar una operación
- Operaciones válidas:
 - **wait:** Bloquea al proceso hasta que otro proceso lo despierte
 - **notify / notify_all:** Despierta a *un proceso / todos los procesos* esperando que se cumpla una condición

Proceso N°1

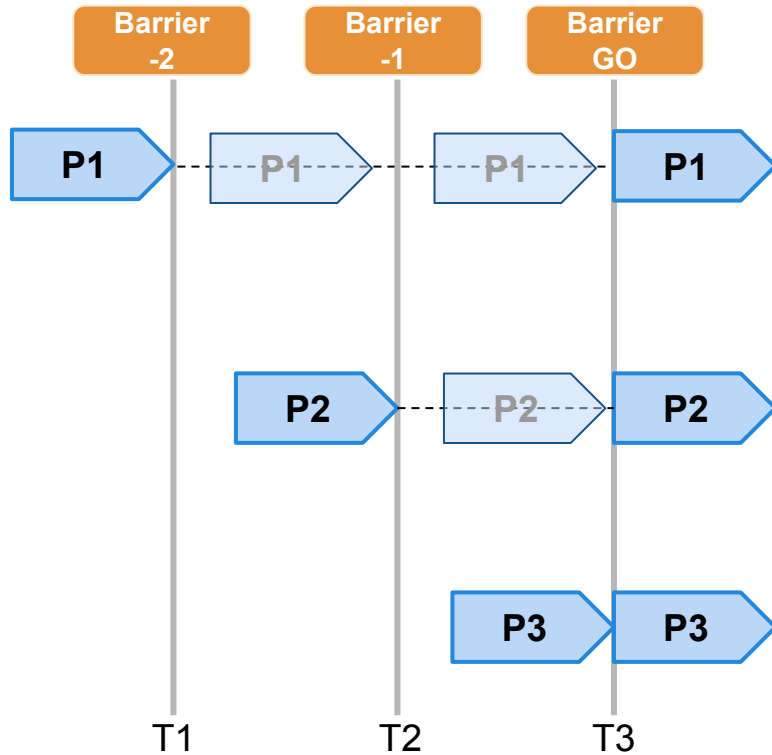
```
cv.acquire()
while not an_item_is_available():
    cv.wait()
get_an_available_item()
cv.release()
```

Proceso N°2

```
cv.acquire()
make_an_item_available()
cv.notify() / cv.notify_all()
cv.release()
```

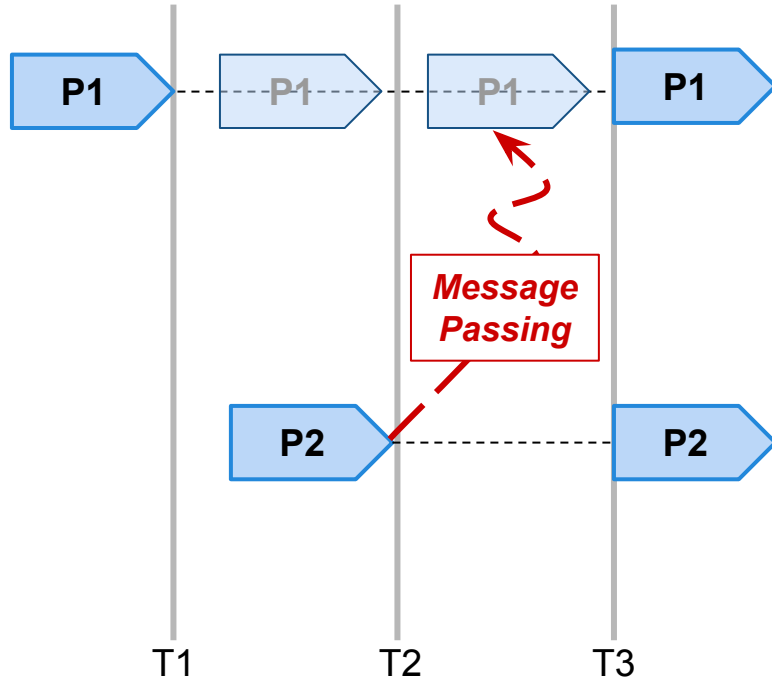



Mecanismos de Sincronización | Barrera





Mecanismos de Sincronización | Rendezvous





Ejercicio

- N *threads/procesos* deben ejecutar M tareas
- Cada *thread/proceso* ejecuta su tarea y espera a que sus pares terminen de hacer lo mismo
- Cuando todos los *threads/procesos* hayan terminado de ejecutar una ronda de tareas, proceden a ejecutar una nueva ronda



Ejercicio

- N *threads/procesos* deben ejecutar M tareas
- Cada *thread/proceso* ejecuta su tarea y espera a que sus pares terminen de hacer lo mismo
- Cuando todos los *threads/procesos* hayan terminado de ejecutar una ronda de tareas, proceden a ejecutar una nueva ronda
- Resolver utilizando la abstracción **BlockingQueue**

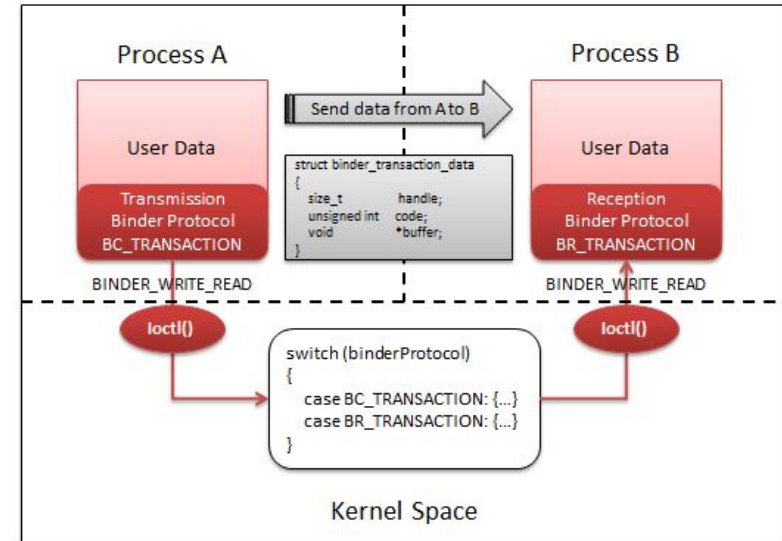
Agenda



- ☐ Conceptos
- ☐ Mecanismos de sincronización
- ☒ **IPCs**
- ☐ Problemas clásicos
- ☐ Comunicaciones



- Permiten la comunicación entre dos o más procesos
- Provistos por el SO
- Creación y destrucción exceden la vida del proceso
 - Usuario es responsable de la vida de los mismos
 - Proceso *Launcher* y *Terminator* para administrar la vida de los mismos
- Usualmente identificados por nombre
- En Linux todos los IPCs son vistos como diferentes **tipos de archivos**





<i>Mecanismo de sincronización</i>	<i>IPC</i>
<i>Semáforo</i>	<i>Semáforo</i>
<i>??</i>	<i>Shared Memory</i>
<i>Monitor</i>	<i>File Lock</i>
<i>Barrera</i>	<i>??</i>
<i>Rendezvous</i>	<i>Signal</i>
	<i>Queue</i>
	<i>Pipes / Fifos</i>
	<i>Sockets</i>



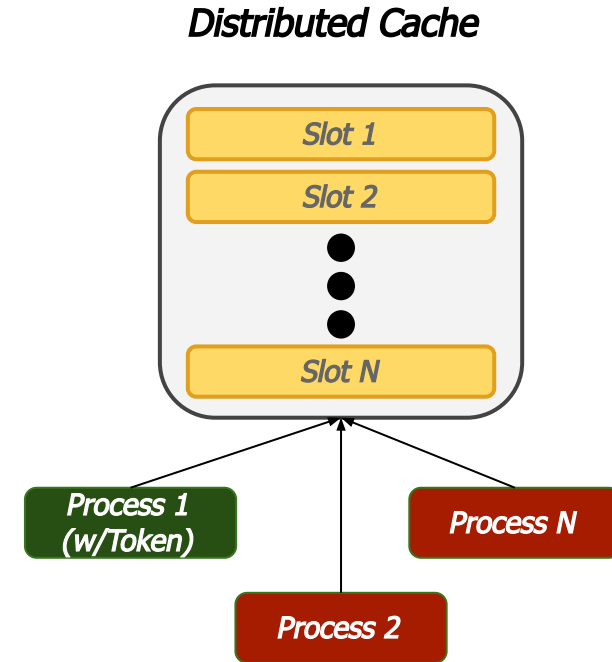
IPCs | Signals

- Existen 31 tipos distintos (kill -l)
- Cada proceso decide cuales handlear (Ej. [libcURL](#) y [SIGALRM](#))
- SIGSTOP y SIGKILL son la excepción
- Ejemplos de signals estándar
 - SIGINT y SIGTERM: *Graceful Quit*
 - SIGSEGV: Problemas en la memoria
 - SIGABRT: Code assertions
- Propagación de *signals* en *threads*. ([Masks setting](#))



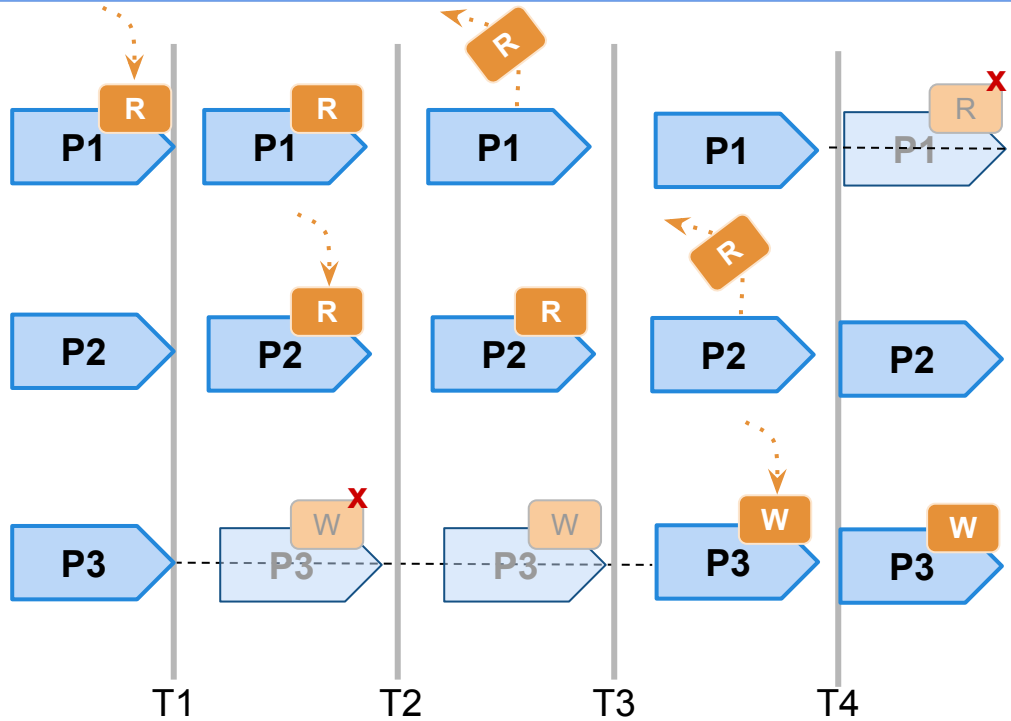


- Mecanismo provisto por el SO (Linux) para compartir recursos
- Abstracción inexistente en threads: heap entre dos threads de un mismo proceso es compartido
- Su tamaño se define al ser creada
- Mutex es necesario **solo si** dos procesos no pueden acceder a la memoria al mismo tiempo (e.g. shared counter)





IPCs | File Locks





IPCs | File Locks

- Control de acceso a un file descriptor

```
int flock(int fd, int operation);
```

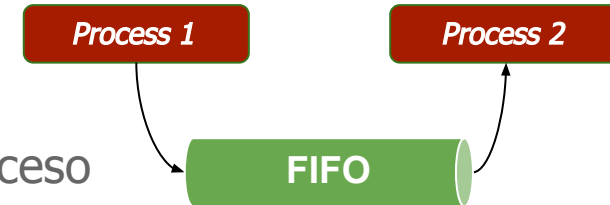
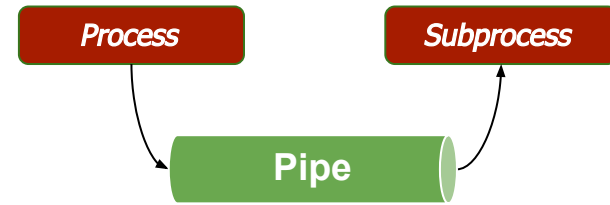
- Existen dos tipos:
 - *Shared lock (R)*: Read only lock.
Múltiples read locks permitidos
 - *Exclusive lock (W)*: RW lock.
Sólo un exclusive lock a la vez por File





IPCs | Pipes y Fifos

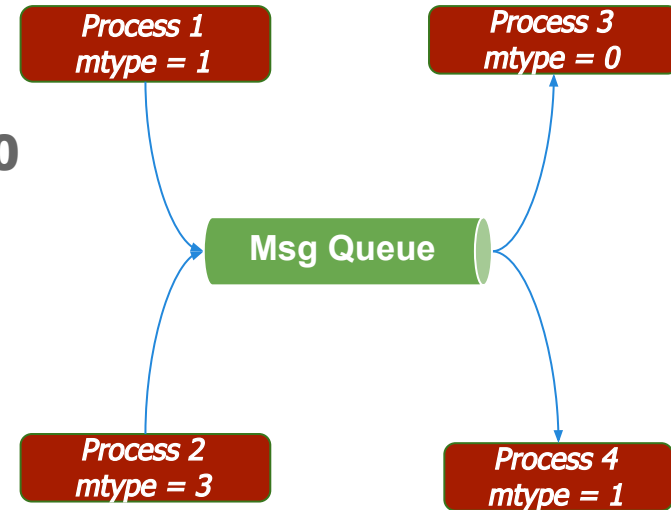
- Pasaje de información directa entre 2 procesos
- Linux: API de un archivo para la escritura/lectura
- Unnamed Pipes (Pipes)
 - Comunicación entre procesos padre e hijo
 - Dejan de existir al finalizar el proceso
- Named Pipes (FIFO)
 - Comunicación entre dos procesos cualesquiera
 - Viven en el SO por lo cual excede la vida del proceso
- ¿Cuál es el tamaño de un pipe?





IPCs | Message Queues (System V)

- Procesos escriben / reciben bloques de bytes
- Campo **mtype**
 - Identifica el tipo de mensaje
 - Sender debe enviar mensajes con **mtype** > 0
 - Receptor con **mtype** = 0 recibe mensajes sin importar el **mtype**
 - Caso esotérico: Receptor con **mtype** < 0
- Mensajes leídos son removidos de la cola
- Buffer size definido durante la creación



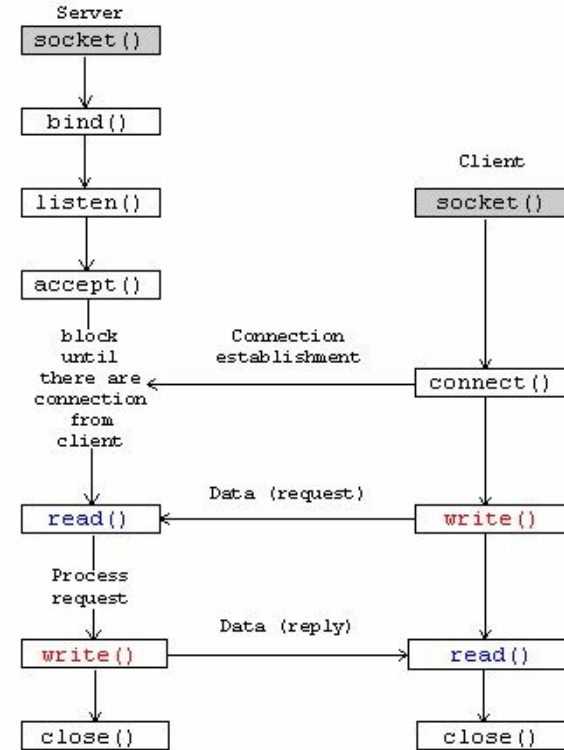


IPCs | Sockets

- Permite comunicar dos procesos a través de un canal de comunicación (endpoint)

```
int socket(int domain, int type, 0);
```

- Domain
 - AF_UNIX - Unix socket
 - AF_INET / AF_INET6 - Network Socket
- Type (Protocolos de comunicación)
 - SOCK_DGRAM => UDP
 - SOCK_STREAM => TCP
 - SOCK_RAW => ??



<https://stackoverflow.com/questions/27014955/socket-connect-vs-bind>



Ejercicio

- N *threads/procesos* deben generar M mensajes
- Cada *thread/proceso* genera un mensaje y espera a que sus pares terminen de hacer lo mismo
- Cuando todos los *threads/procesos* hayan terminado de generar una ronda de mensajes procederán a generar uno nuevo
- **Resolver utilizando IPCs**
 - Shared Memory / Semaphores
 - Message Queues

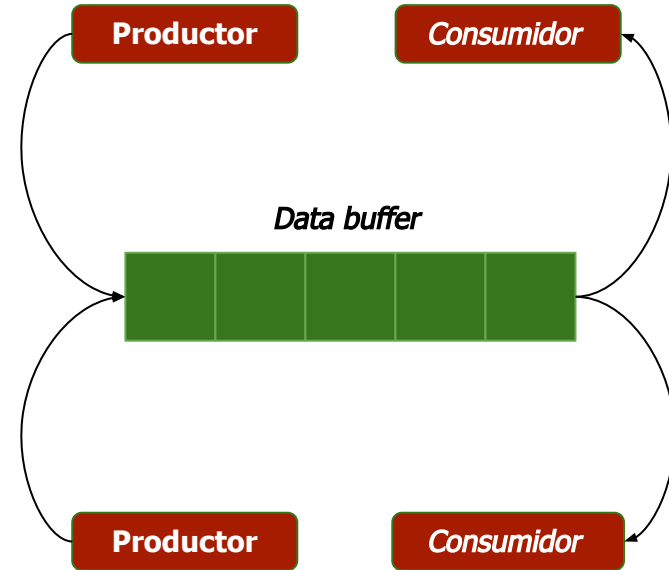


- Conceptos
- Mecanismos de sincronización
- IPCs
- **Problemas clásicos**
- Paralelización de tareas
- Comunicaciones



Problemas clásicos | Productor Consumidor

- Productores agregan paquetes en el buffer
- Consumidores extraen paquetes del buffer
- Situaciones de bloqueo
 - Productor intenta agregar un paquete cuando el buffer está lleno
 - Consumidor intenta extraer un paquete cuando el buffer está vacío
- Acceso al buffer **debe** ser sincronizado
- El buffer es acotado o infinito?

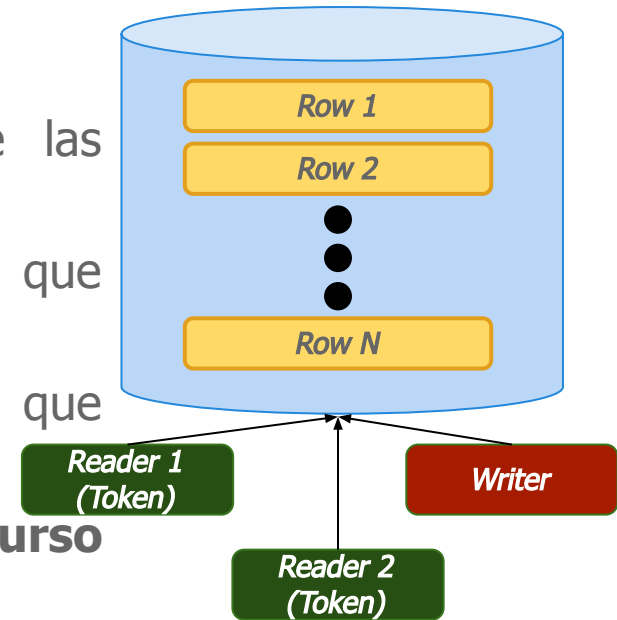




Problemas clásicos | Lectores Escritores

- Procesos intentan acceder a una memoria compartida
- Dos tipos de procesos (Lectores y Escritores)
- Tipos de problemas definidos en función de las propiedades *fairness* y *starvation*
 - *Prioridad Lectores* : Escritores esperan a que lectores liberen recurso compartido
 - *Prioridad Escritores* : Lectores esperan a que Escritores liberen recurso compartido
 - Lectores y Escritores acceden a **recurso compartido por tiempo limitado**

Database Access





- Barbero dormilón
- Filósofos comensales
- Fumadores de cigarrillos

Agenda



- Conceptos
- Mecanismos de sincronización
- IPCs
- Problemas clásicos
- **Comunicaciones**

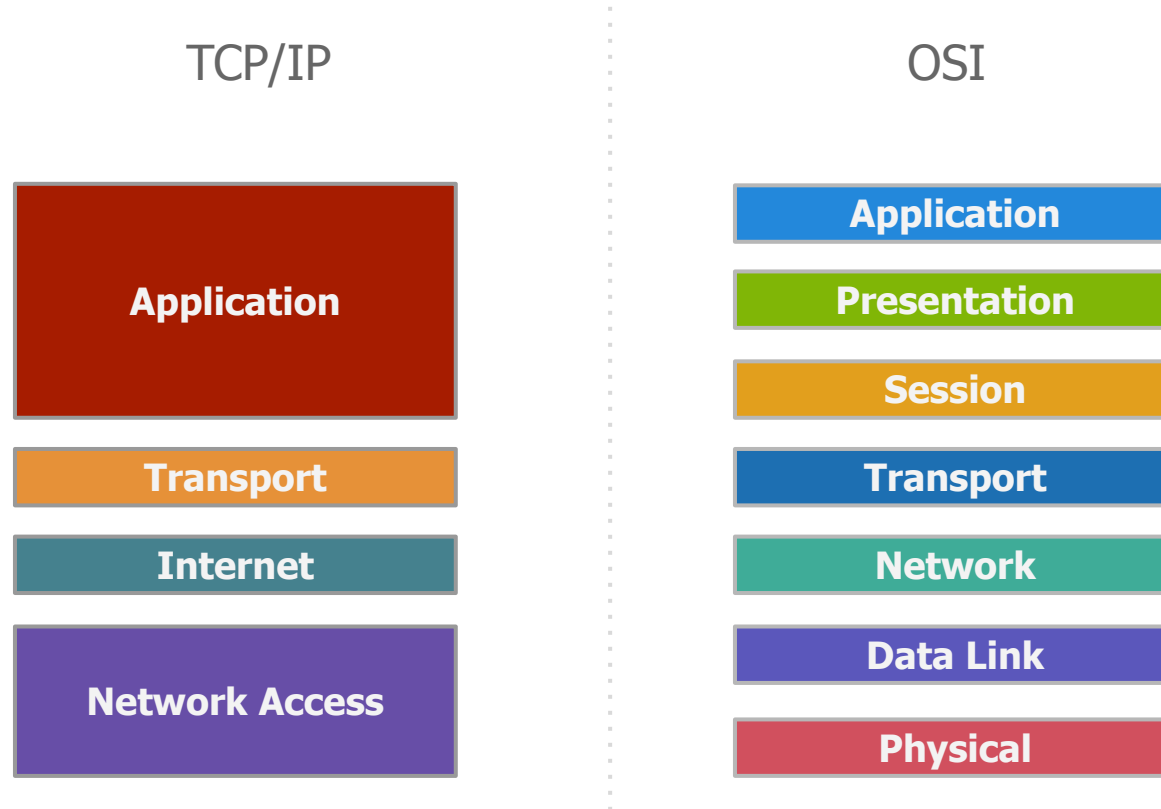


Application	Aplicaciones de usuario, representación de datos
Transport	Comunicación punto a punto
Internet	Lógica de transmisión de datos sobre la red
Network Access	Transferencia física confiable, libre de errores.



Application	Aplicaciones de usuario
Presentation	Representación de datos
Session	Manejo de conexiones y sesión
Transport	Transferencia confiable, libre de errores
Network	Establecer, mantener y terminar conexiones. Transmisión
Data Link	Sincronización, control de errores y envío de frames
Physical	Manejo del medio físico para transmitir bits

Comunicaciones | Relaciones entre OSI y TCP/IP



Estructura de Paquetes | IP



Ver.	IHL	Type of Service	Total Length	
Identification			Flags	Frag. Offset
TTL	Protocol		Header Checksum	
Source Address				
Dest. Address				
Options				
Data (variable)				

32 bits



Estructura de Paquetes | TCP y UDP

TCP

- Orientado a conexión
- Asegura entrega y orden

Source Port		Dest. Port	
Sequence Number			
Ack. Number			
Data Offset	Reserved	Flags	Window (sliding win.)
Checksum		Urgent Pointer	
Options			Padding
Data (variable)			

32 bits

UDP

- Orientado a datos
- Sin garantías: *best effort*

Source Port	Dest. Port
Length	Checksum
Data (variable)	

32 bits



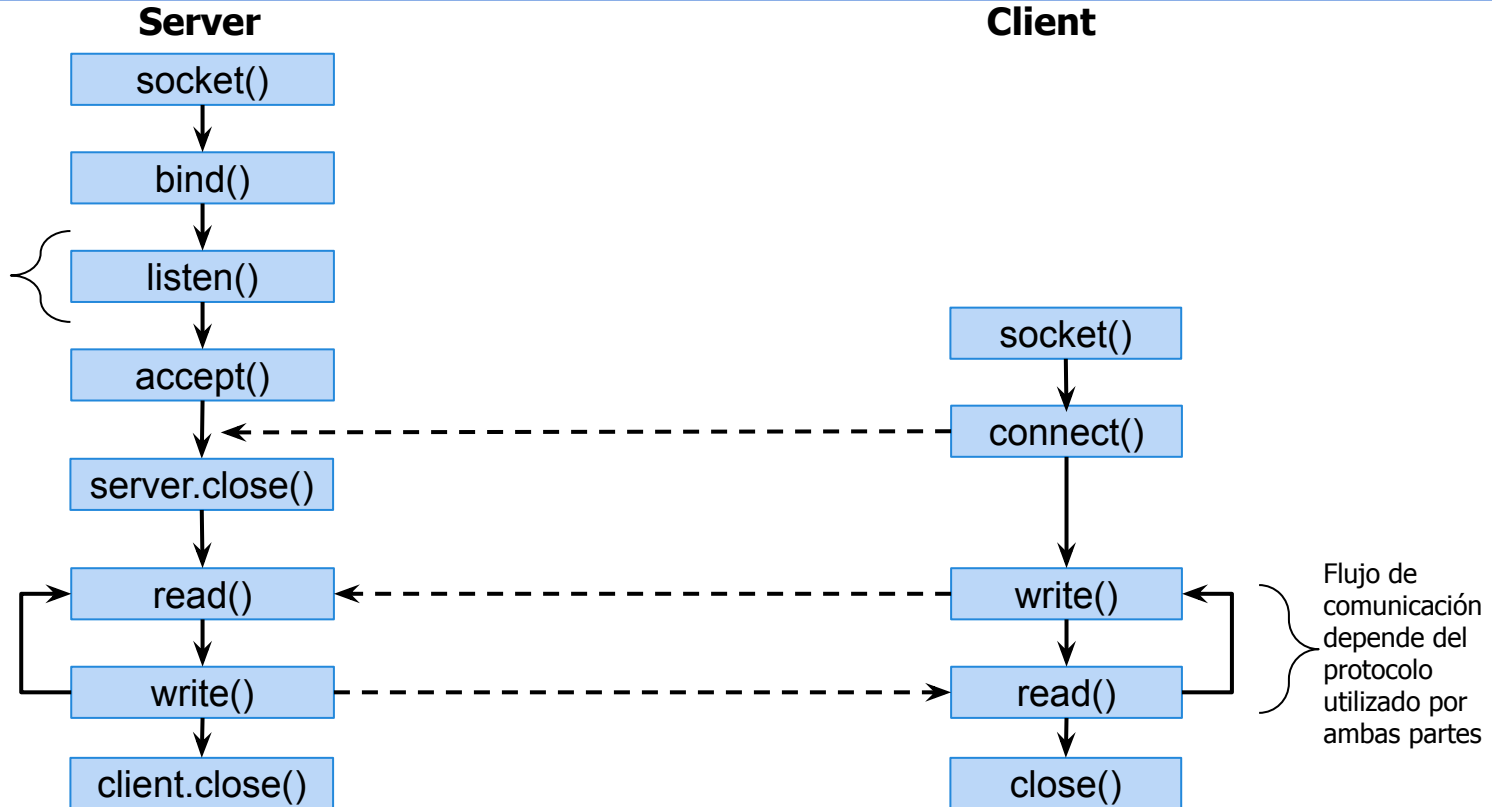
Sockets | Primitivas

- **socket**(domain: **int**, type: **int**, protocol: **int**): **int**
- **bind**(fd: **int**, addr: struct sockaddr*, addrlen: **int**): **int**
- **listen**(fd: **int**, backlog: **int**): **int**
- **accept**(fd: **int**, addr: struct sockaddr*, addrlen: **int**): **int**
- **connect**(fd: **int**, addr: struct sockaddr*, addrlen: **int**): **int**
- **send**(fd: **int**, buf: **void***, buflen: **int**, flags: **int**): **int**
- **receive**(fd: **int**, buf: **void***, buflen: **int**, flags: **int**): **int**
- **close**(fd: **int**): **int**

Sockets | Flujo de uso TCP

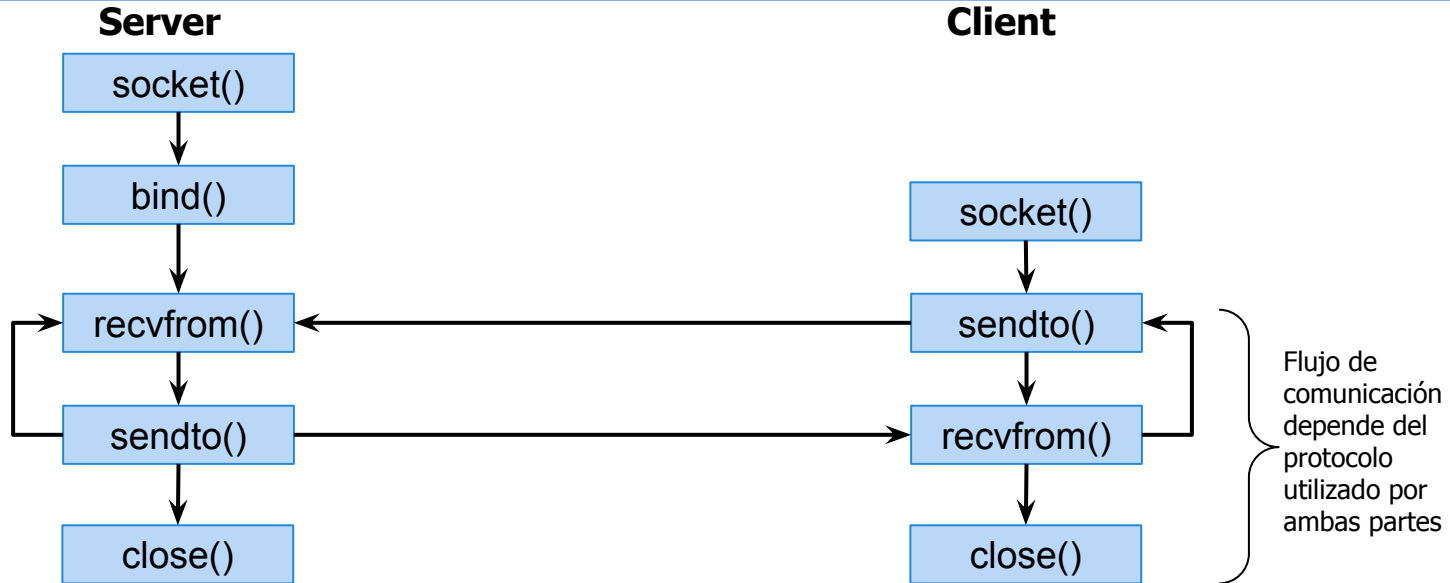


Listen backlog
aplica sobre
conexiones
pendientes, no
establecidas.



Flujo de
comunicación
depende del
protocolo
utilizado por
ambas partes

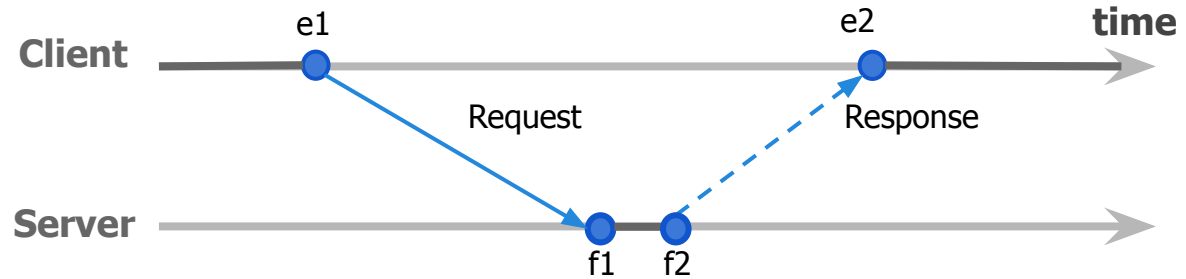
Sockets | Flujo de uso UDP



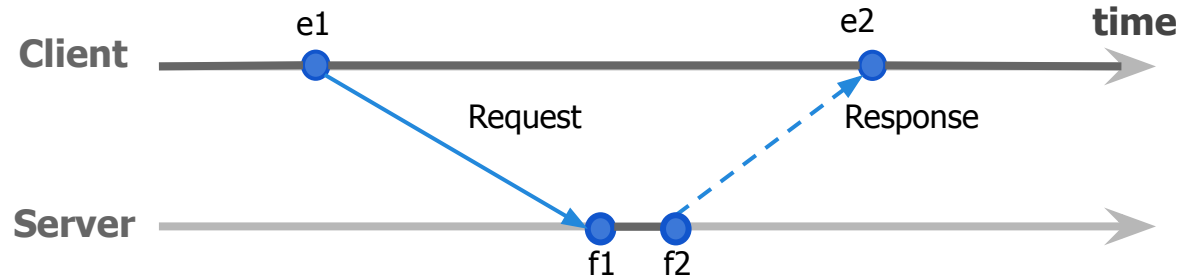


Mensajes Sincrónicos vs Asincrónicos

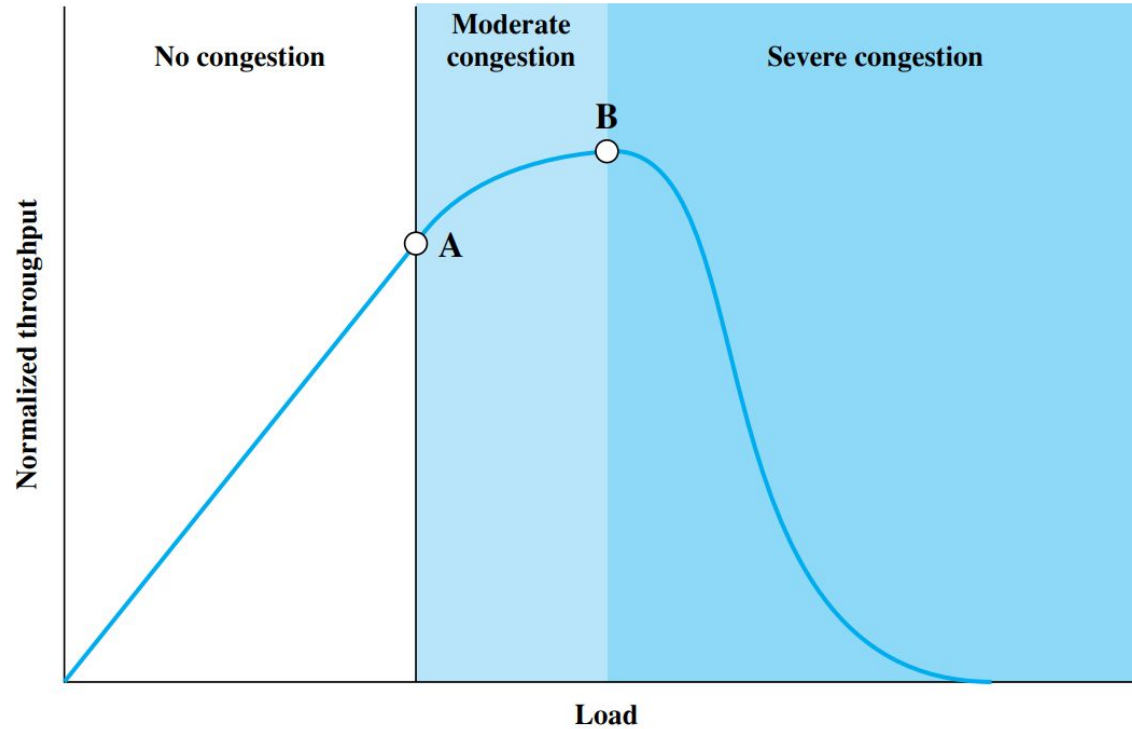
Sincrónico



Asincrónico

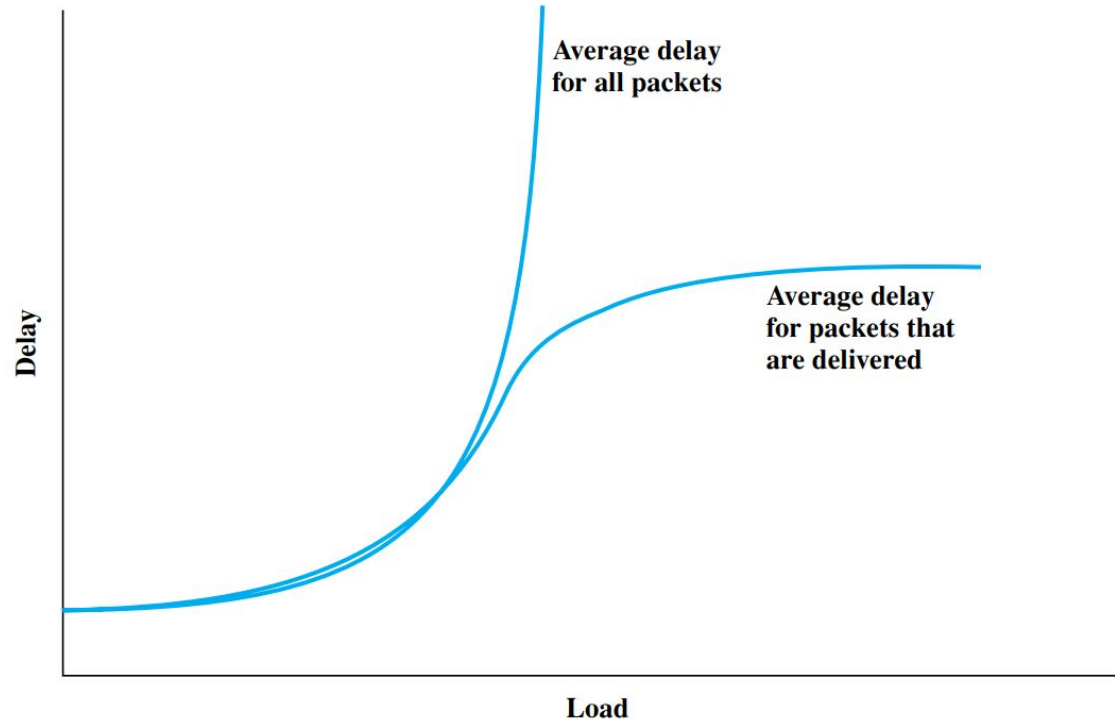


Congestión de Red | *Throughput y Throttling*



Source: W. Stallings. Data and Computer Communications. 8th edition.

Congestión de Red | *Delay*





- McCool M., Robison A. D., Reinders J., Structured Parallel Programming Patterns for Efficient Computation, 2012, Elsevier-Morgan Kaufmann.
 - Capítulo 1: Introduction
 - Capítulo 2: Background
- Ben-Ari, M. Principles of Concurrent and Distributed Programming, 2nd. Ed. Addison Wesley, 2006.
 - Capítulo 2: The concurrent programming abstraction
 - Capítulo 3: The mutual exclusion problem
 - Capítulo 4: Semaphores
 - Capítulo 5: Monitors